

Anna University Regulation: 2021

CS3591- Computer Networks

III Year/V

Semester Lab

Manual

Prepared by
S.Adlin Femil
AP/CSE

Ex.No.1 Learn to use commands like tcpdump, netstat, ifconfig, nslookup and trace route. Capture ping and trace route PDUs using a network protocol analyzer and examine

AIM:

To study the basic networking commands like tcpdump, netstat, ifconfig, nslookup and trace route. Capture ping and trace route PDUs using a network protocol analyzer and examine.

COMMANDS:

C:\>arp -a: ARP is short form of address resolution protocol, It will show the IP address of your computer along with the IP address and MAC address of your router.

C:\>hostname: This is the simplest of all TCP/IP commands. It simply displays the name of your computer.

C:\>ipconfig: The ipconfig command displays information about the host (the computer your sitting at)computer TCP/IP configuration.

C:\>ipconfig /all: This command displays detailed configuration information about your TCP/IP connection including Router, Gateway, DNS, DHCP, and type of Ethernet adapter in your system.

C:\>Ipconfig /renew: Using this command will renew all your IP addresses that you are currently (leasing) borrowing from the DHCP server. This command is a quick problem solver if you are having connection issues, but does not work if you have been configured with a static IP address.

C:\>Ipconifg /release: This command allows you to drop the IP lease from the DHCP server.

C:\>ipconfig /flushdns: This command is only needed if you're having trouble with your networks DNS configuration. The best time to use this command is after network configuration frustration sets in, and you really need the computer to reply with flushed.

C:\>nbtstat -a: This command helps solve problems with NetBIOS name resolution. (Nbt stands for NetBIOS over TCP/IP)

C:\>netdiag: Netdiag is a network testing utility that performs a variety of network diagnostic tests, allowing you to pinpoint problems in your network. Netdiag isn't installed by default,

but can be installed from the Windows XP CD after saying no to the install. Navigate to the CD ROM drive letter and open the support\tools folder on the XP CD and click the setup.exe icon in the support\tools folder.

C:\>netstat: Netstat displays a variety of statistics about a computers active TCP/IP connections. This tool is most useful when you're having trouble with TCP/IP applications such as HTTP, and FTP.

C:\>nslookup: Nslookup is used for diagnosing DNS problems. If you can access a resource by specifying an IP address but not it's DNS you have a DNS problem.

C:\>pathping: Pathping is unique to Window's, and is basically a combination of the Ping and Tracert commands. Pathping traces the route to the destination address then launches a 25 second test of each router along the way, gathering statistics on the rate of data loss along each hop.

C:\>ping: Ping is the most basic TCP/IP command, and it's the same as placing a phone call to your best friend. You pick up your telephone and dial a number, expecting your best friend to reply with "Hello" on the other end. Computers make phone calls to each other over a network by using a Ping command. The Ping commands main purpose is to place a phone call to another computer on the network, and request an answer. Ping has 2 options it can use to place a phone call to another computer on the network. It can use the computers name or IP address.

C:\>route: The route command displays the computers routing table. A typical computer, with a single network interface, connected to a LAN, with a router is fairly simple and generally doesn't pose any network problems. But if you're having trouble accessing other computers on your network, you can use the route command to make sure the entries in the routing table are correct.

C:\>tracert: The tracert command displays a list of all the routers that a packet has to go through to get from the computer where tracert is run to any other computer on the internet.

RESULT:

Thus the above list of tcpdump, netstat, ifconfig, nslookup and trace route. Capture ping and trace route PDUs has been studied and executed successfully.

Ex.No.2 Write a HTTP web client program to download a web page using TCP sockets

Aim:

To write a program in java to create a HTTP web client program to download a web page using TCP sockets.

Algorithm:

1. Start the program
2. Read the file to be downloaded from webpage
3. To download an image, use java URL class which can be found under java.net package.
4. The file is downloaded from server and is stored in the current working directory.
5. Stop the program

Program

Download.java

```
import java.io.*;
import java.net.URL;
public class Download
{
    public static void main(String[] args) throws Exception
    {
        try
        {
            String fileName = "digital_image_processing.jpg";
            String website = "http://tutorialspoint.com/java_dip/images/"+fileName;
            System.out.println("Downloading File From: " + website);
            URL url = new URL(website);
            InputStream inputStream = url.openStream();
            OutputStream outputStream = new FileOutputStream(fileName);
            byte[] buffer = new byte[2048];
            int length = 0;
            while ((length = inputStream.read(buffer)) != -1)
            {
                System.out.println("Buffer Read of length: " + length);
                outputStream.write(buffer, 0, length);
            }
        }
    }
}
```

```

        }
        inputStream.close();
        outputStream.close();
    }
    catch(Exception e)
    {
        System.out.println("Exception: " + e.getMessage());
    }
}
}

```

Output

F:\Abarna\Lab>javac Download.java

F:\ Abarna \Lab>java Download

Downloading File From: http://tutorialspoint.com/java_dip/images/digital_image_processing.jpg

Buffer Read of length: 2048

Buffer Read of length: 2048

Buffer Read of length: 2048

Buffer Read of length: 2048

Buffer Read of length: 2048

Buffer Read of length: 1097

Buffer Read of length: 2048

Buffer Read of length: 2048

Buffer Read of length: 1744

Buffer Read of length: 2048

Buffer Read of length: 2048

Buffer Read of length: 2048

Buffer Read of length: 2048

Buffer Read of length: 2048

Buffer Read of length: 1440

Buffer Read of length: 2048

Buffer Read of length: 2048

Buffer Read of length: 2048

Buffer Read of length: 2048

Buffer Read of length: 2048

Buffer Read of length: 2048

Buffer Read of length: 2048

Buffer Read of length: 2048

Buffer Read of length: 2048

Buffer Read of length: 548

Buffer Read of length: 2048

Buffer Read of length: 2048

Buffer Read of length: 2048

Buffer Read of length: 2048

Buffer Read of length: 2048

Buffer Read of length: 2048

Buffer Read of length: 2048

Buffer Read of length: 2048

Buffer Read of length: 1863

The downloaded file (Stored in the current working directory)



Result:

Thus created a program in java for a HTTP web client program to download a web page using TCP sockets is written and executed successfully.

Ex.No.3a APPLICATION USING TCP SOCKETS - ECHO CLIENT AND ECHO SERVER

Aim:

To write a program in Java to implement an applications using TCP Sockets like echo client and echo server

Algorithm

1. Start the Program
2. In Server
 - a) Create a server socket and bind it to port.
 - b) Listen for new connection and when a connection arrives, accept it.
 - c) Read the data from client.
 - d) Echo the data back to the client.
 - e) Close all streams.
 - f) Close the server socket.
 - g) Stop.
3. In Client
 - a) Create a client socket and connect it to the server's port number.
 - b) Send user data to the server.
 - c) Display the data echoed by the server.
 - d) Close the input and output streams.
 - e) Close the client socket.
 - f) Stop.
4. Stop the program

Program

Server.java

```
import java.net.*;
import java.lang.*;
import java.io.*;
public class Server
{
    public static final int PORT = 4000;
    public static void main( String args[])
```

```

{
    ServerSocket sersock = null;
    Socket sock = null;
    try
    {
        sersock = new ServerSocket(PORT);
        System.out.println("Server Started :"+sersock);
        try
        {
            sock = sersock.accept();
            System.out.println("Client Connected :"+ sock);
            DataInputStream ins = new DataInputStream(sock.getInputStream());
            System.out.println(ins.readLine());
            PrintStream ios = new PrintStream(sock.getOutputStream());
            ios.println("Hello from server");
            ios.close();
            sock.close();

        }
        catch(SocketException se)
        {
            System.out.println("Server Socket problem "+se.getMessage());
        }
    }
    catch(Exception e)
    {
        System.out.println("Couldn't start " + e.getMessage());
    }
    System.out.println(" Connection from : " + sock.getInetAddress());
}

```



```
}
```

Client.java

```
import java.lang.*;
```

```
import java.io.*;
```

```
import java.net.*;
```

```
import java.net.InetAddress;
```

```
class client
```

```
{
```

```
    public static void main(String args[])
```

```
    {
```

```
        Socket sock=null;
```

```
        DataInputStream dis=null;
```

```
        PrintStream ps=null;
```

```
        System.out.println(" Trying to connect");
```

```
        try
```

```
        {
```

```
            sock= new Socket(InetAddress.getLocalHost(),Server.PORT);
```

```
            ps= new PrintStream(sock.getOutputStream());
```

```
            ps.println(" Hi from client");
```

```
            DataInputStream is = new DataInputStream(sock.getInputStream());
```

```
            System.out.println(is.readLine());
```

```
        }
```

```
        catch(SocketException e)
```

```
        {
```

```
            System.out.println("SocketException " + e);
```

```
        }
```

```
        catch(IOException e)
```

```
        {
```

```
            System.out.println("IOException " + e);
```

```
        }
```

```
        finally
```

```

        {
            try
            {
                sock.close();
            }
            catch(IOException ie)
            {
                System.out.println(" Close Error :" + ie.getMessage());
            }
        }
    }
}

```

Output

In server window:

F:\ Abarna \Lab>javac Server.java

F:\ Abarna \Lab>java Server

Server Started :ServerSocket[addr=0.0.0.0/0.0.0.0,port=0,localport=4000]

Client Connected :Socket[addr=/192.168.1.18,port=1815,localport=4000]

Hi from client

Connection from : /192.168.1.18

In client window:

F:\ Abarna\Lab>javac client.java

F:\ Abarna\Lab>java client

Trying to connect

Hello from server

Result:

Thus a program in Java implemented an applications using TCP Sockets like echo client and echo server

Ex.No.3b**APPLICATION USING TCP SOCKETS - CHAT****Aim**

To write a program in Java to implement an applications using TCP Sockets like chat.

Algorithm

1. Start the Program
2. In Server
 - a) Create a server socket and bind it to port.
 - b) Listen for new connection and when a connection arrives, accept it.
 - c) Read Client's message and display it
 - d) Get a message from user and send it to client
 - e) Repeat steps 3-4 until the client sends "end"
 - f) Close all streams
 - g) Close the server and client socket
 - h) Stop
3. In Client
 - a) Create a client socket and connect it to the server's port number
 - b) Get a message from user and send it to server
 - c) Read server's response and display it
 - d) Repeat steps 2-3 until chat is terminated with "end" message
 - e) Close all input/output streams
 - f) Close the client socket
 - g) Stop
4. Stop the program

Program**tcpchatserver.java**

```
import java.io.*;
import java.net.*;
class tcpchatserver
{
    public static void main(String args[])throws Exception
    {
```

```

        PrintWriter toClient;
        BufferedReader fromUser, fromClient;
        try
        {
            ServerSocket Srv = new ServerSocket(4000);
            System.out.print("\nServer started\n");
            Socket Clt = Srv.accept();
            System.out.println("Client connected");
            toClient = new PrintWriter(new BufferedWriter(new
            OutputStreamWriter(Clt.getOutputStream())), true);
        fromClient = new BufferedReader(new InputStreamReader(Clt.getInputStream()));
            fromUser = new BufferedReader(new InputStreamReader(System.in));
            String CltMsg, SrvMsg;
            while(true)
            {
                CltMsg= fromClient.readLine();
                if(CltMsg.equals("end"))
                    break;
                else
                {
                    System.out.println("Server      : "+CltMsg);
                    System.out.print("Message to Client : ");
                    SrvMsg = fromUser.readLine();
                    toClient.println(SrvMsg);
                }
            }
            System.out.println("\nClient Disconnected");
            fromClient.close();
            toClient.close();
            fromUser.close();
            Clt.close();

```

```

        Srv.close();
    }
    catch (Exception E)
    {
        System.out.println(E.getMessage());
    }
}
}

```

tcpchatclient.java

```

import java.io.*;
import java.net.*;
class tcpchatclient
{
    public static void main(String args[])throws Exception
    {
        Socket Clt;
        PrintWriter toServer;
        BufferedReader fromUser, fromServer;
        try
        {
            Clt = new Socket(InetAddress.getLocalHost(),4000);
            toServer = new PrintWriter(new BufferedWriter(new
            OutputStreamWriter(Clt.getOutputStream())), true);
            fromServer = new BufferedReader(new InputStreamReader(Clt.getInputStream()));
            fromUser = new BufferedReader(new InputStreamReader(System.in));
            String CltMsg, SrvMsg;
            System.out.println("Type \"end\" to Quit");
            while (true)
            {
                System.out.print("Message to Server : ");
                CltMsg = fromUser.readLine();
            }
        }
    }
}

```

```

        toServer.println(CltMsg);
        if (CltMsg.equals("end"))
            break;
        SrvMsg = fromServer.readLine();
        System.out.println("Client      : " + SrvMsg);
    }
}
catch(Exception E)
{
    System.out.println(E.getMessage());
}
}
}

```

Output

In server window:

F:\ Abarna\ Lab>javac tcpchatserver.java

F:\ Abarna\ Lab>java tcpchatserver

Server started

Client connected

Server : hai

Message to Client : hello

Client Disconnected

In client window:

F:\ Abarna\ Lab>javac tcpchatclient.java

F:\ Abarna\ Lab>java tcpchatclient

Type "end" to Quit

Message to Server : hai

Client : hello

Message to Server : end

Result:

Thus a program in Java implemented an application using TCP Sockets like chat.

Ex:NO 4

Simulation of DNS using UDP sockets

Aim:

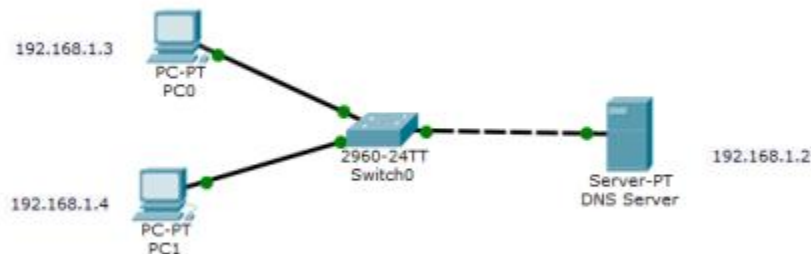
To stimulate DNS using UDP sockets in packet tracer

Algorithm:

- i)Open Cisco packet tracer
- ii)Place Switch, laptop ,and pc and assign IP address for laptop and PC
- iii)No default gateway is needed because there is no router
- iv)Open command prompt and give the command to display ip address
- v)Click Desktop and provide the IP address
- vi)Click Server and select DNS and assign the name and address
- vii)Click http and select index.html to edit the message
- viii)Click desktop and select web browser then assign the same name www.mypage.com then click go button and the mesaage will display on the screen

Steps:

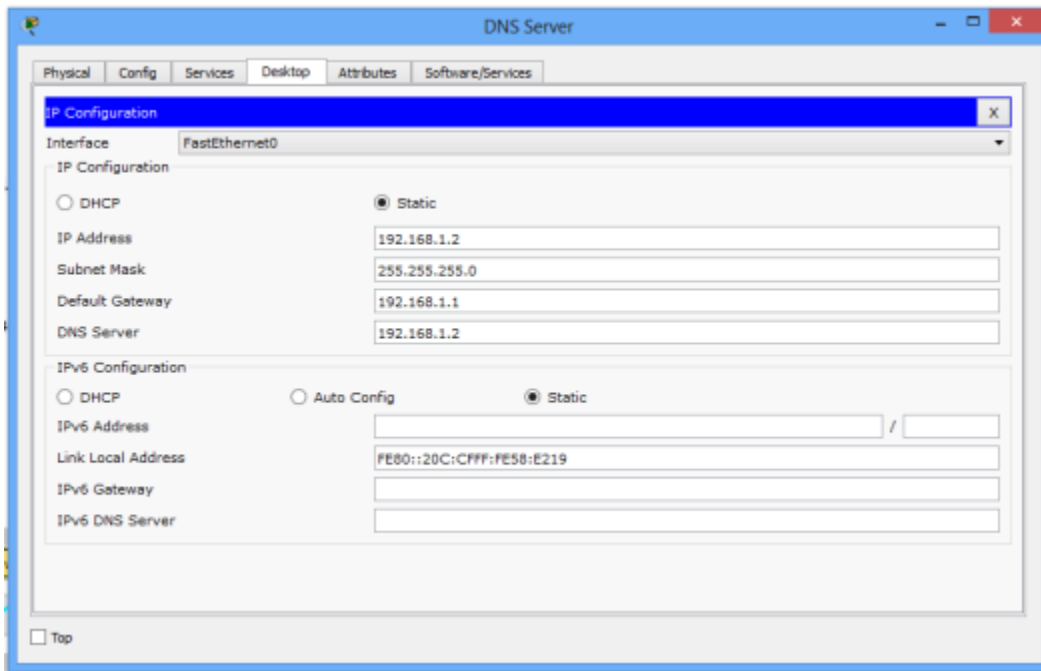
1.Build the network topology.



2.Configure static IP addresses on the PCs and the server.

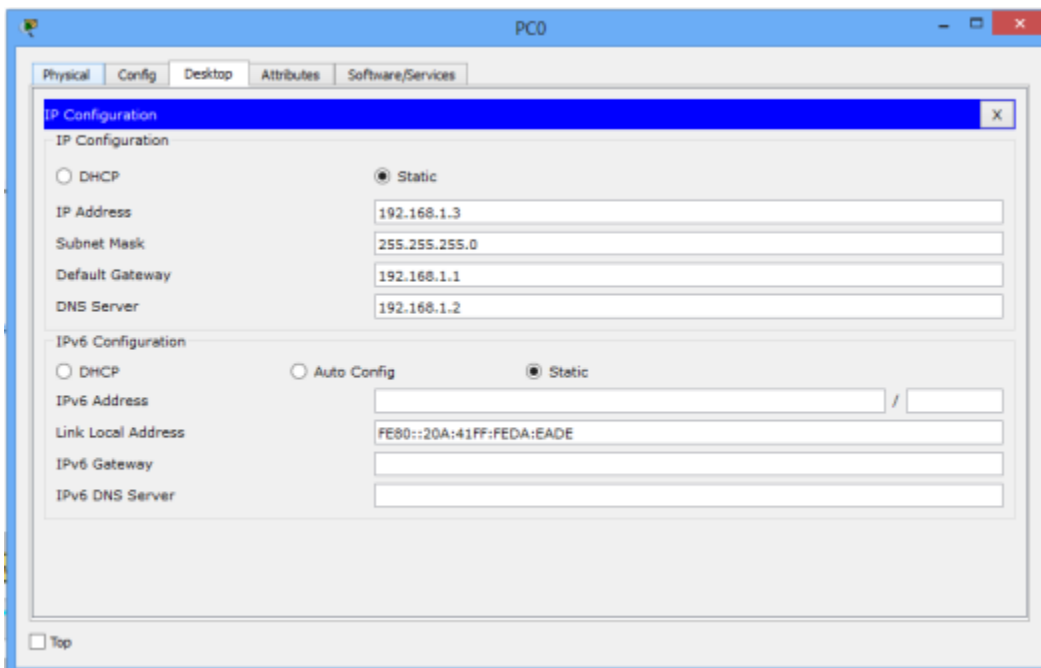
Server

IP address: 192.168.1.2 **Subnet mask:** 255.255.255.0 **Default gateway:** 192.168.1.1 **DNS Server:** 192.168.1.2



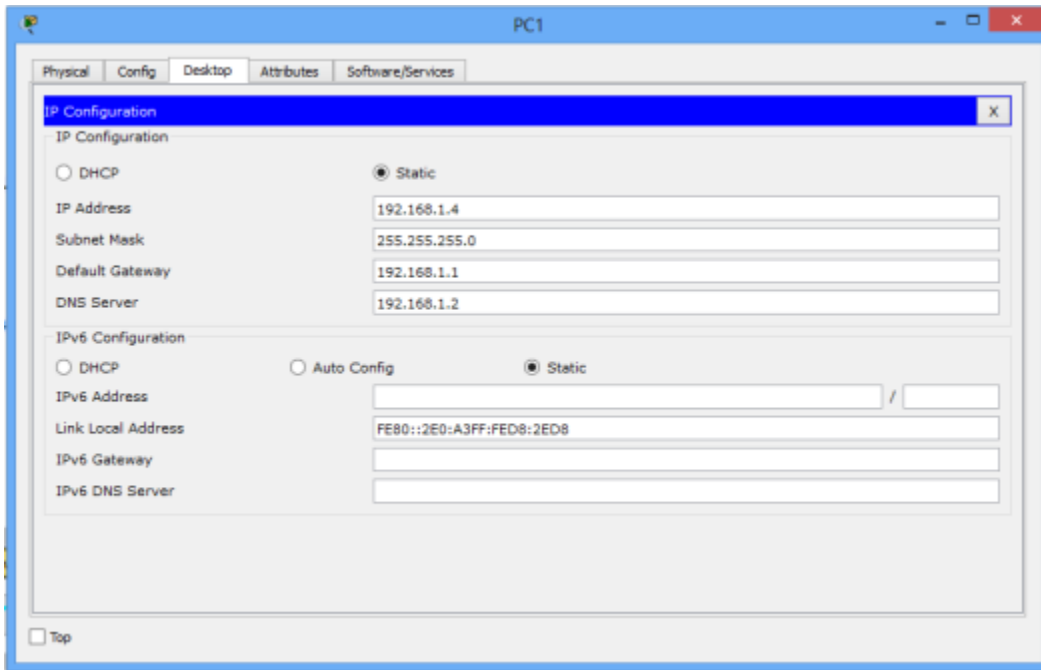
PC0

IP add: 192.168.1.3 **Subnet mask:** 255.255.255.0 **Default gateway:** 192.168.1.1 **DNS server:** 192.168.1.2



PC1

IP address: 192.168.1.4 **Subnet mask:** 255.255.255.0 **Default gateway:** 192.168.1.1 **DNS Server:** 192.168.1.2



3.Configure DNS service on the generic server.

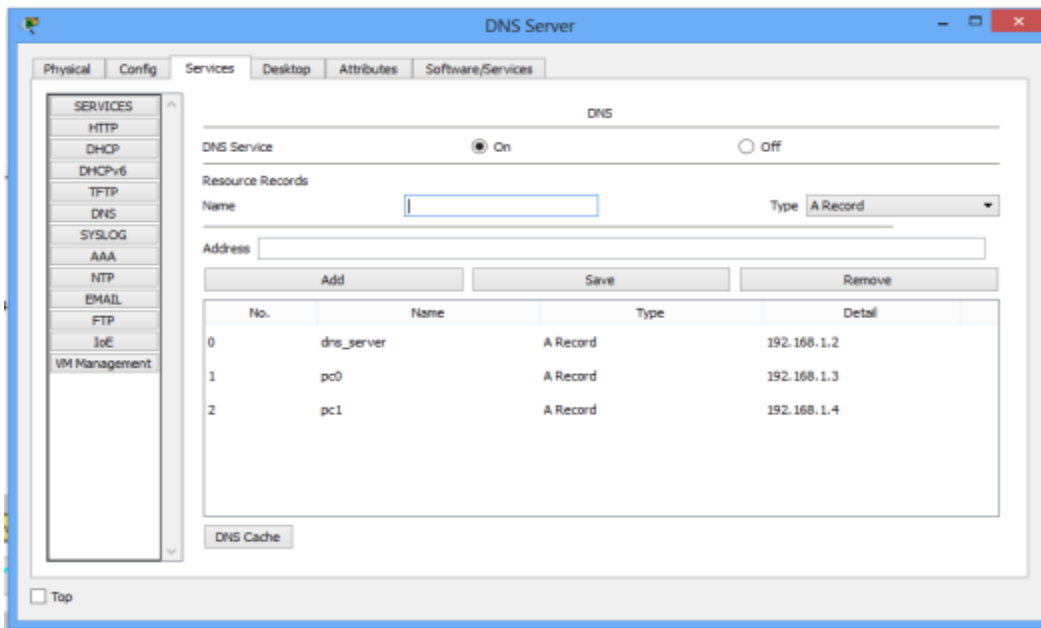
To do this, click on the server, then Click on **Services** tab. Click on **DNS server** from the menu. First turn **ON** the DNS service, then define **names** of the hosts and their corresponding **IP addresses**.

For example, to specify the DNS entry for PC0: In the **name** and **address** fields, type:

Name: PC0 **Address:** 192.168.1.3

Click on **add** then **save**. Repeat this for the PC1 and the server.

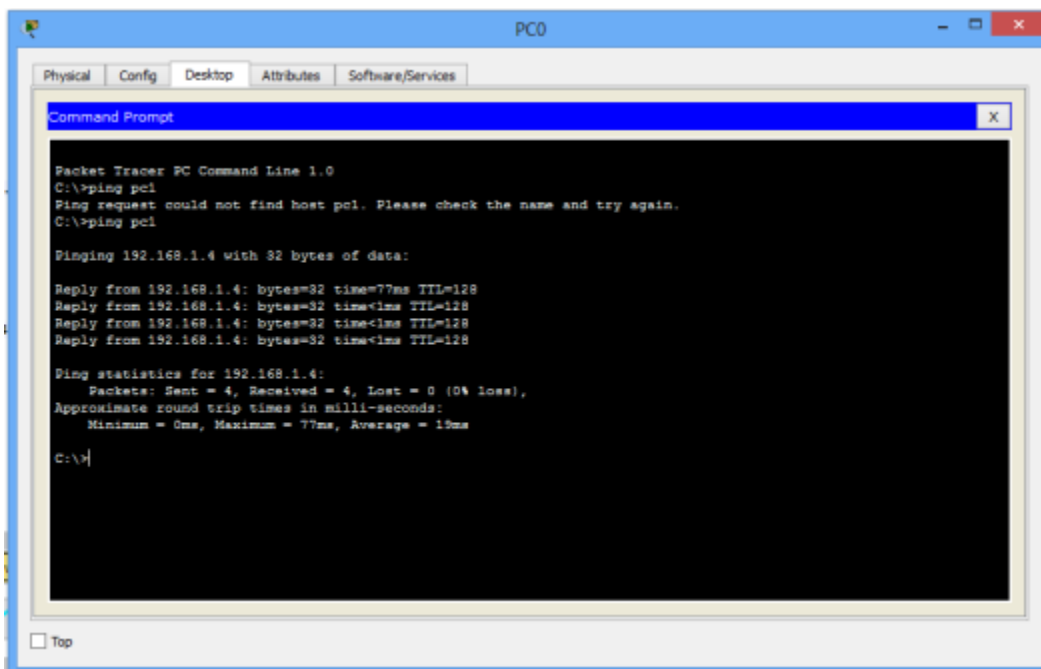
Once you're done, your DNS entries will look like this:



Finally,

4. Test **domain name – IP resolution**. Ping the hosts from one another using their names instead of their IP addresses. If the DNS service is turned on and all IP configurations are okay, then ping should work.

For example, ping PC1 from PC0. Ping should be successful.



Result:

Thus the simulation of DNS using UDP socket was executed successfully

Ex.No.5 USE A TOOL LIKE WIRESHARK TO CAPTURE PACKETS AND EXAMINE THE PACKETS

Aim :

To implement the code to capture packets and examine the packets using wireshark.

Procedure:

1. A network packet analyzer will try to capture network packets and tries to display that packet data as detailed as possible.
2. Identifying and analyzing protocols.
3. Identifying source & destination of traffic.

Program :

```
import sys

from scapy.all import *

# Define the packet capturing

functiondef

packet_handler(packet):

    print(packet.show())

# Capture packets on the network

interfacesniff(iface='eth0',

prn=packet_handler)
```

Output:

```
####[ Ethernet ]####
dst  = 00:11:22:33:44:55
src  = 66:77:88:99:00:11
type = IPv4 ####[ IP ]####
version = 4
ihl  = 5
tos  = 0x0
len  = 1500
id   = 52786
flags= DF
frag = 0
ttl  = 128
proto= tcp
checksum= 0xe877
src  = 192.168.1.10
dst  = 216.58.194.78
```

```
\options \ ###[ TCP ]###  
sport= 54321  
dport= http  
seq  = 3665840854  
ack  = 2542707921  
dataofs = 5  
reserved = 0  
flags = FA  
window= 8760  
chksum= 0x9d6c  
urgptr= 0  
options = [('MSS', 1460), ('SAckOK', ''), ('Timestamp', (12576855, 2413)), ('NOP', None),  
('WScale', 7)] ###[ Raw ]###  
load = "
```

Result:

Thus the program was executed successfully using tool Wireshark to capture packet and examine the packet.

Ex.No.6

SIMULATION OF ARP /RARP PROTOCOLS

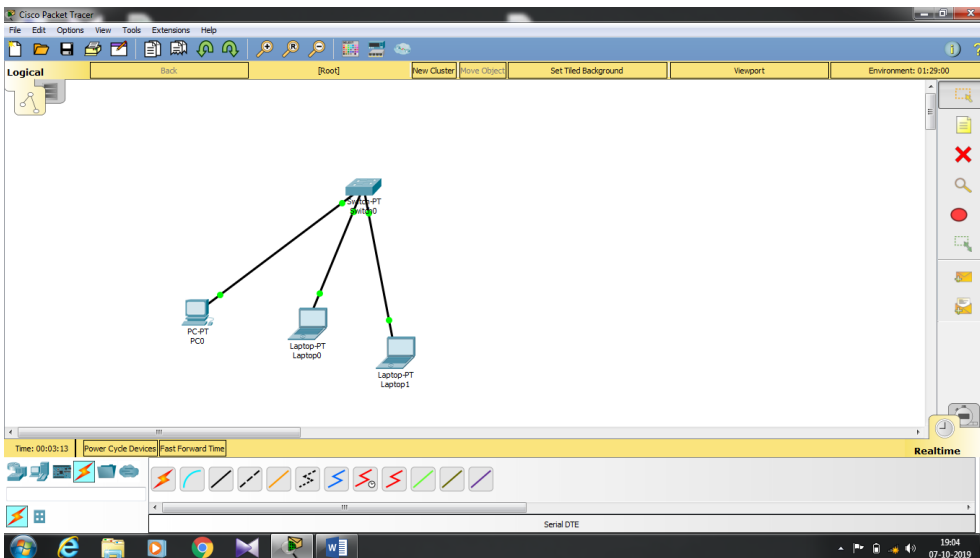
Aim:

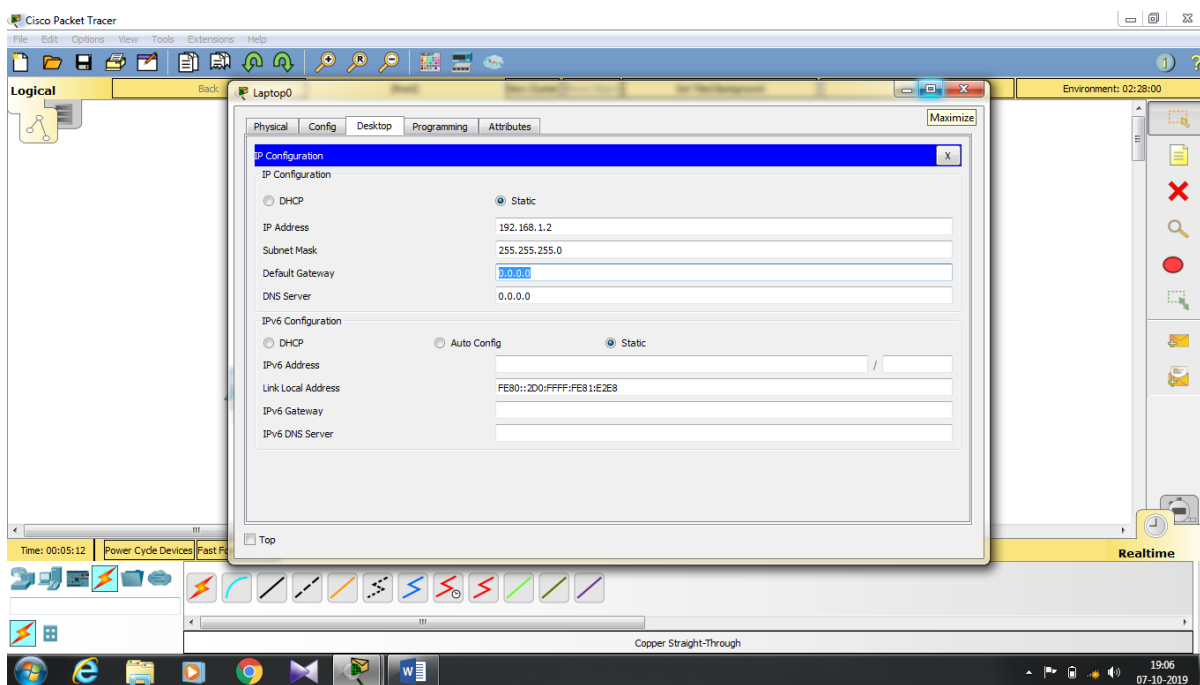
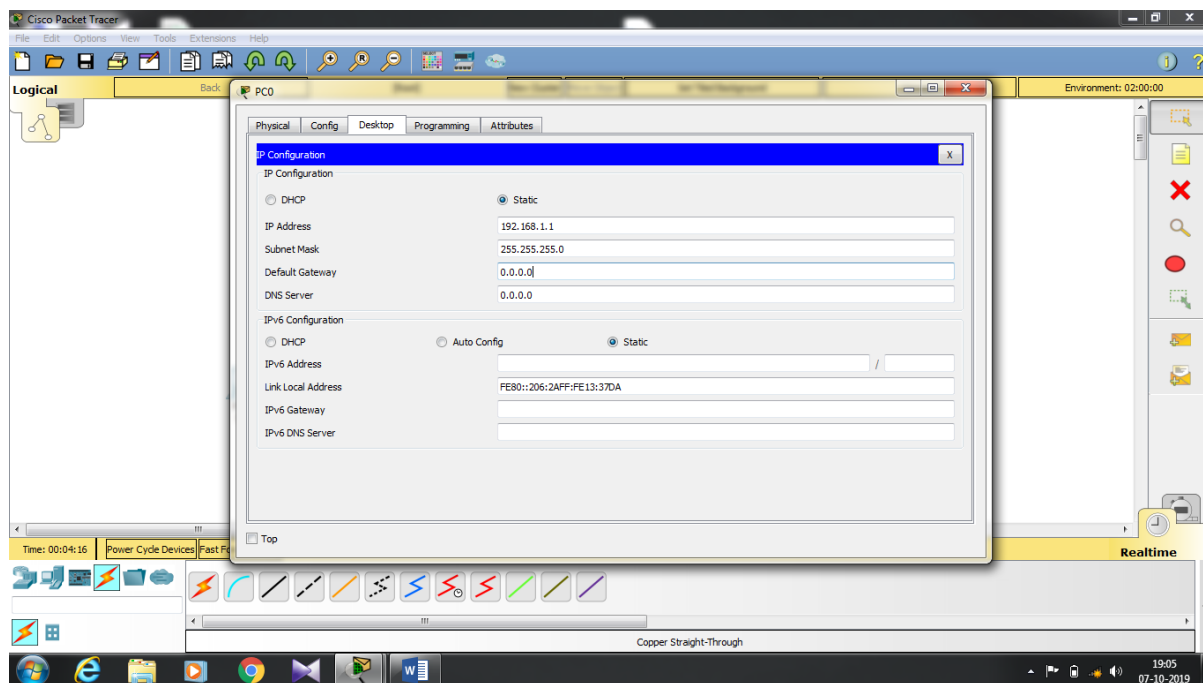
To write a code for simulating ARP and RARP using cisco packet tracer

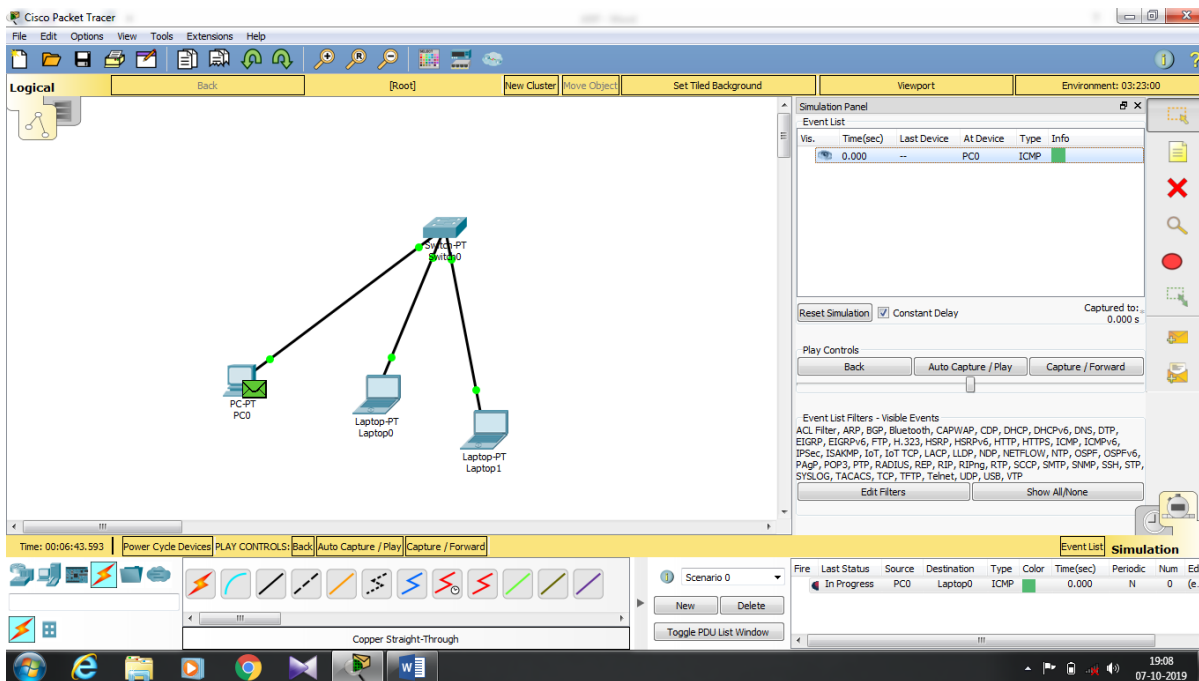
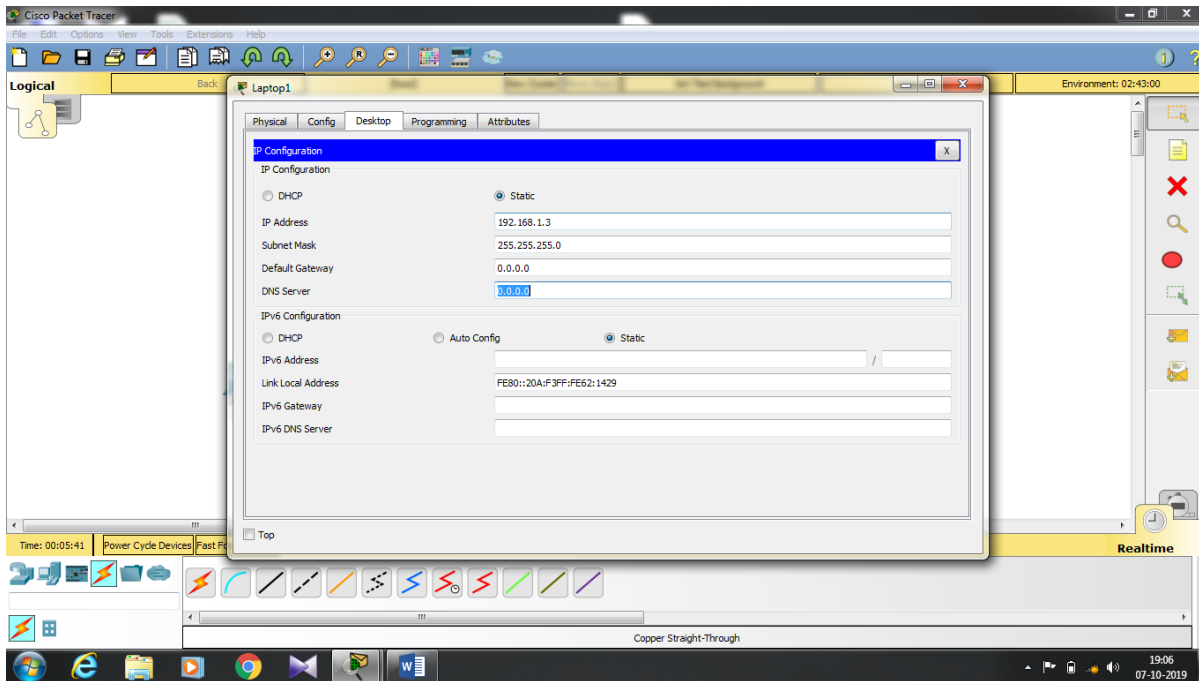
ALGORITHM:

- i)Open Cisco packet tracer
- ii)Place Switch, laptop, and pc and assign IP address for laptop and PC
- iii)No default gateway is needed because there is no router
- iv)Open command prompt and give the command to display ip address
- v)Forward the packets from one node to another node

Steps:







Cisco Packet Tracer

File Edit Options View Tools Extensions Help

Logical [Back] [Root] New Cluster Move Object Set Tiled Background Viewport Environment: 03:32:30

Simulation Panel

Event List

Vis.	Time(sec)	Last Device	At Device	Type	Info
	0.000	--	PC0	ICMP	
	0.001	PC0	Switch0	ICMP	

Reset Simulation ☒ Constant Delay Captured to: 0.001 s

Play Controls: Back Auto Capture / Play Capture / Forward

Event List Filters - Visible Events

ACL Filter, ARP, BGP, Bluetooth, CAPWAP, CDP, DHCP, DHCPv6, DNS, DTP, EIGRP, EIGRPv6, FTP, H.323, HSRP, HSRPv6, HTTP, HTTPS, ICMP, ICMPv6, IPsec, ISAKMP, IoT, IoT TCP, LACP, LLDP, NDP, NETFLOW, NTP, OSPF, OSPFv6, PAgP, POP3, PTP, RADIUS, REP, RIP, RIPng, RTP, SCCP, SMTP, SNMP, SSH, STP, SYSLOG, TACACS, TCP, TFTP, Telnet, UDP, USB, VTP

Edit Filters Show All/None

Time: 00:06:43.594 Power Cycle Devices PLAY CONTROLS: Back Auto Capture / Play Capture / Forward

Scenario 0 New Delete Toggle PDU List Window

Fire Last Status Source Destination Type Color Time(sec) Periodic Num Edit

In Progress PC0 Laptop0 ICMP 0.000 N 0

19:08 07-10-2019

Cisco Packet Tracer

File Edit Options View Tools Extensions Help

Logical [Back] [Root] New Cluster Move Object Set Tiled Background Viewport Environment: 03:43:30

Simulation Panel

Event List

Vis.	Time(sec)	Last Device	At Device	Type	Info
	0.000	--	PC0	ICMP	
	0.001	PC0	Switch0	ICMP	
	0.002	Switch0	Laptop0	ICMP	

Reset Simulation ☒ Constant Delay Captured to: 0.002 s

Play Controls: Back Auto Capture / Play Capture / Forward

Event List Filters - Visible Events

ACL Filter, ARP, BGP, Bluetooth, CAPWAP, CDP, DHCP, DHCPv6, DNS, DTP, EIGRP, EIGRPv6, FTP, H.323, HSRP, HSRPv6, HTTP, HTTPS, ICMP, ICMPv6, IPsec, ISAKMP, IoT, IoT TCP, LACP, LLDP, NDP, NETFLOW, NTP, OSPF, OSPFv6, PAgP, POP3, PTP, RADIUS, REP, RIP, RIPng, RTP, SCCP, SMTP, SNMP, SSH, STP, SYSLOG, TACACS, TCP, TFTP, Telnet, UDP, USB, VTP

Edit Filters Show All/None

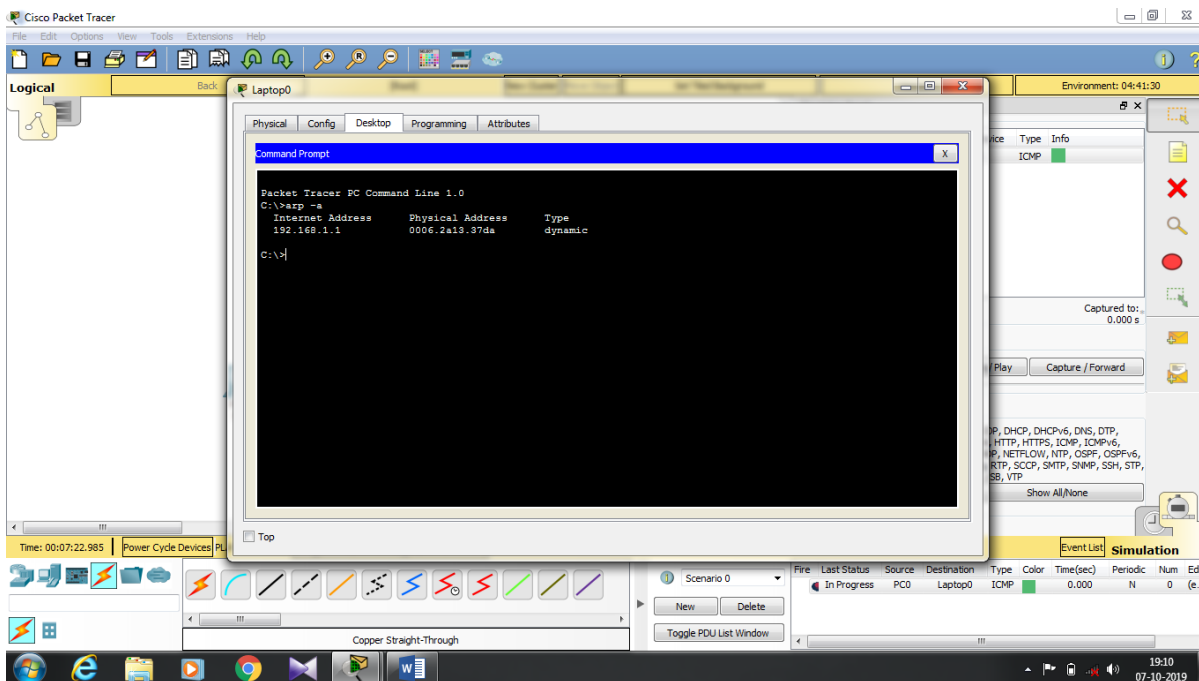
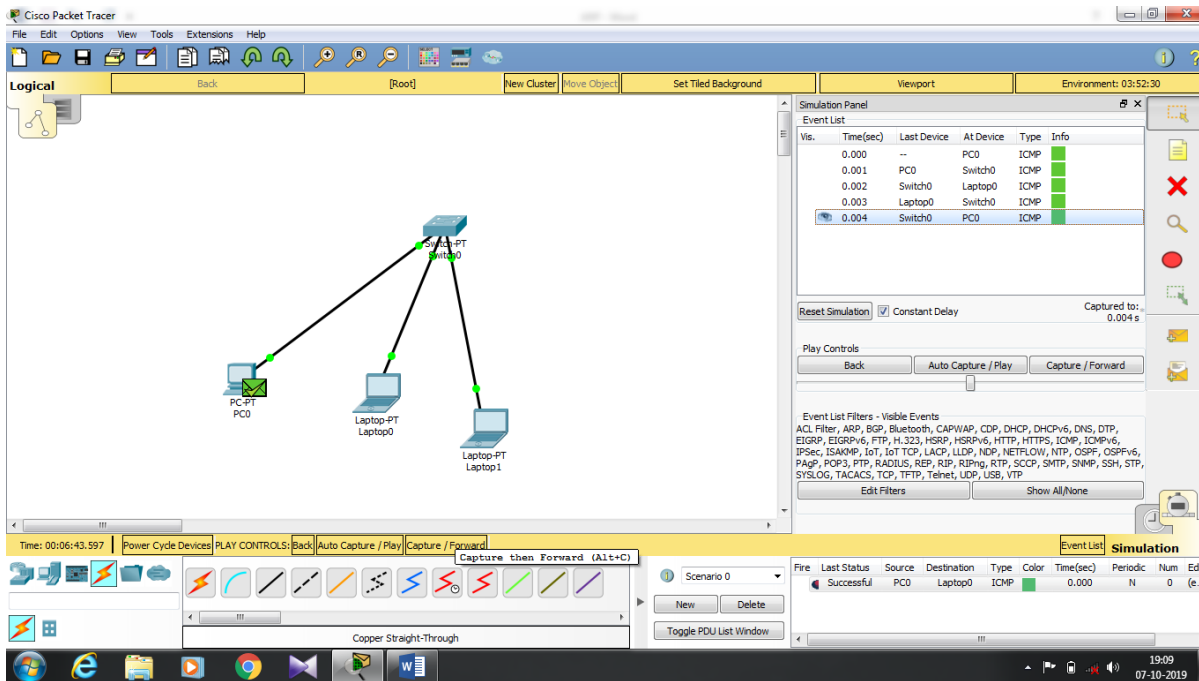
Time: 00:06:43.595 Power Cycle Devices PLAY CONTROLS: Back Auto Capture / Play Capture / Forward

Scenario 0 New Delete Toggle PDU List Window

Fire Last Status Source Destination Type Color Time(sec) Periodic Num Edit

In Progress PC0 Laptop0 ICMP 0.000 N 0

19:08 07-10-2019



Result:

Thus the message transmit from ARP to RARP in simulation using cisco packet tracer is executed successfully.

Ex:NO: 7 Study of Network simulator (NS) and Simulation of Congestion Control Algorithms using NS.

Aim:

To study about Network simulator (NS) and Simulation of Congestion Control Algorithms using NS.

Procedure:

In computer network research, **network simulation** is a technique whereby a software program models the behavior of a network by calculating the interaction between the different network entities (routers, switches, nodes, access points, links etc.). Most simulators use discrete event simulation - the modeling of systems in which state variables change at discrete points in time. The behavior of the network and the various applications and services it supports can then be observed in a test lab; various attributes of the environment can also be modified in a controlled manner to assess how the network / protocols would behave under different conditions.

A **network simulator** is software that predicts the behavior of a computer network. Since communication networks have become too complex for traditional analytical methods to provide an accurate understanding of system behavior, network simulators are used. In simulators, the computer network is modeled with devices, links, applications etc. and the network performance is reported. Simulators come with support for the most popular technologies and networks in use today such as 5G, Internet of Things (IoT), Wireless LANs, mobile ad hoc networks, wireless sensor networks, vehicular ad hoc networks, cognitive radio networks, LTE etc.

Most of the commercial simulators are GUI driven, while some network simulators are CLI driven. The network model / configuration describes the network (nodes, routers, switches, links) and the events (data transmissions, packet error etc.). Output results would include network level metrics, link metrics, device metrics etc. Further, drill down in terms of simulations trace files would also be available. Trace files log every packet, every event that occurred in the simulation and are used for analysis. Most network simulators use discrete event simulation, in which a list of pending "events" is stored, and those events are processed in order, with some events triggering future events—such as the event of the arrival of a packet at one node triggering the event of the arrival of that packet at a downstream node.

Network emulation

Network emulation allows users to introduce real devices and applications into a test network (simulated) that alters packet flow in such a way as to mimic the behavior of a live network. Live traffic can pass through the simulator and be affected by objects within the simulation.

The typical methodology is that real packets from a live application are sent to the emulation server

(where the virtual network is simulated). The real packet gets 'modulated' into a simulation packet. The simulation packet gets demodulated into a real packet after experiencing effects of loss, errors, delay, jitter etc., thereby transferring these network effects into the real packet. Thus it is as-if the real packet flowed through a real network but in reality it flowed through the simulated network.

Emulation is widely used in the design stage for validating communication networks prior to deployment.

List of network simulators

There are both free/open-source and proprietary network simulators available. Examples of notable network simulators / emulators include:

- Packet Tracer
- OPNET
- Tetcos NetSim
- NS2 / NS3

Uses of network simulators

Network simulators provide a cost-effective method for

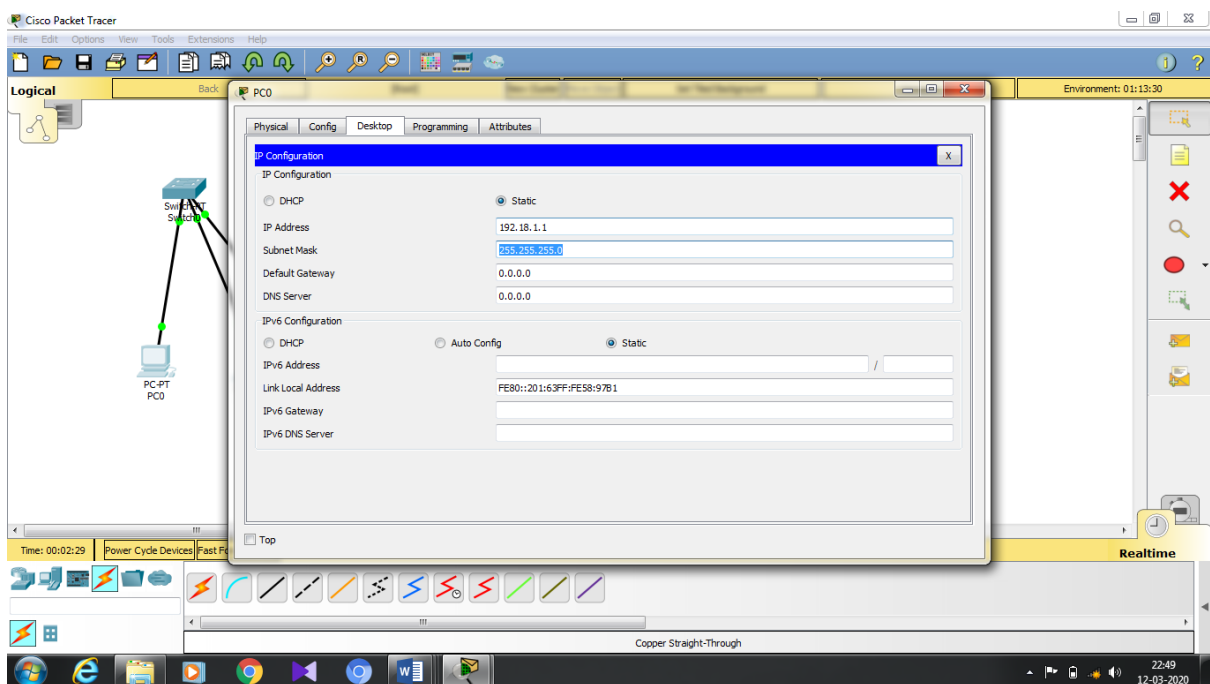
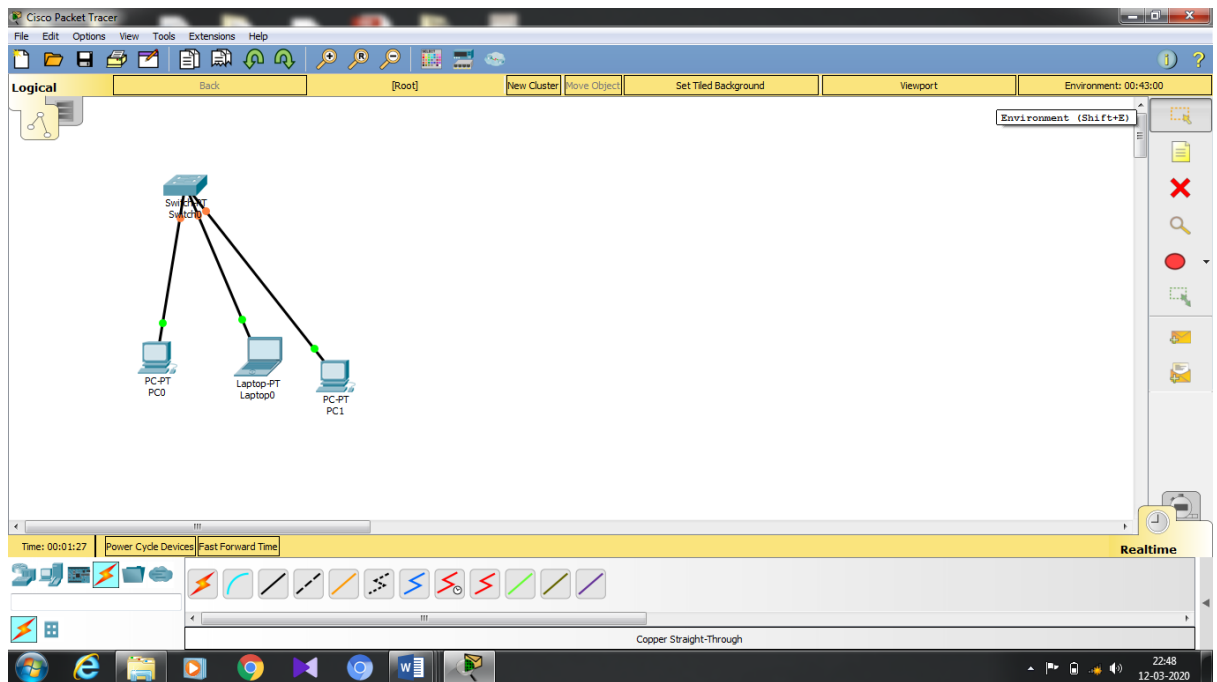
- Network design validation for enterprises / data centers / sensor networks etc.
- Network R & D (More than 70% of all Network Research paper reference a network simulator)
- Defense applications such as HF / UHF / VHF Radio based MANET Radios, Tactical data links etc.
- LTE, LTE Advanced, IOT, VANET simulations
- Satellite communication
- Optimization of cache placement on large virtual content distribution networks
- Education - Lab experimentation and R & D. Most universities use a network simulator for teaching / R & D since its too expensive to buy hardware equipment

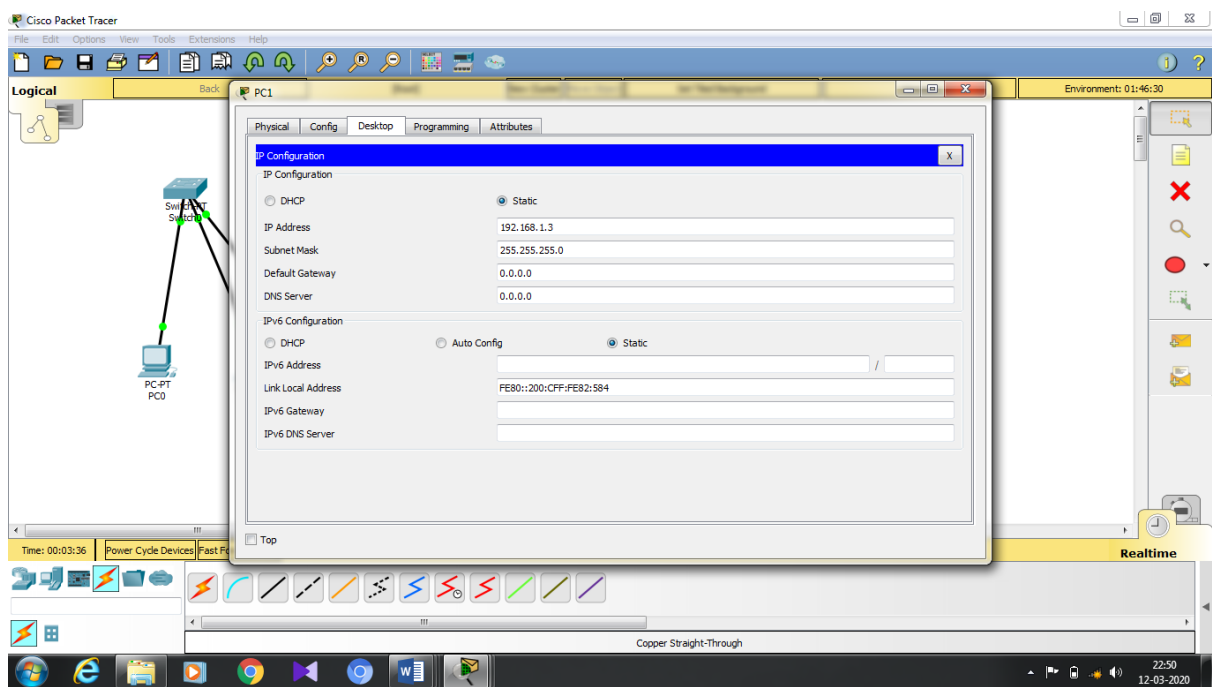
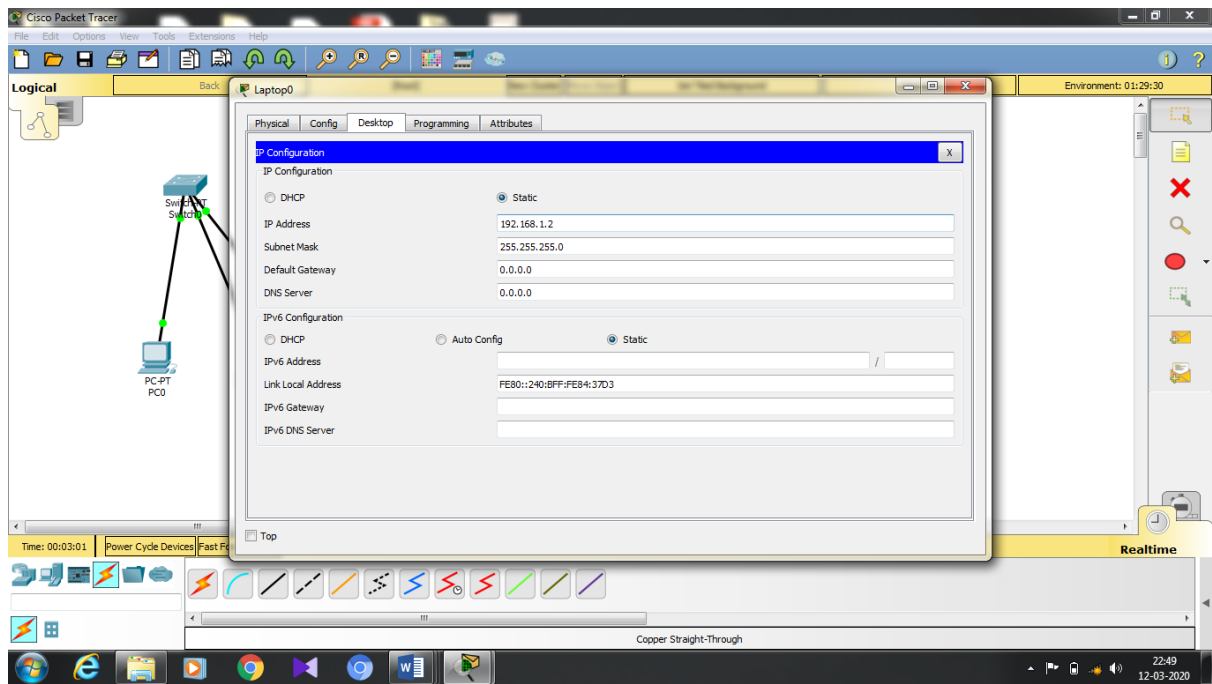
There are a wide variety of network simulators, ranging from the very simple to the very complex. Minimally, a network simulator must enable a user to

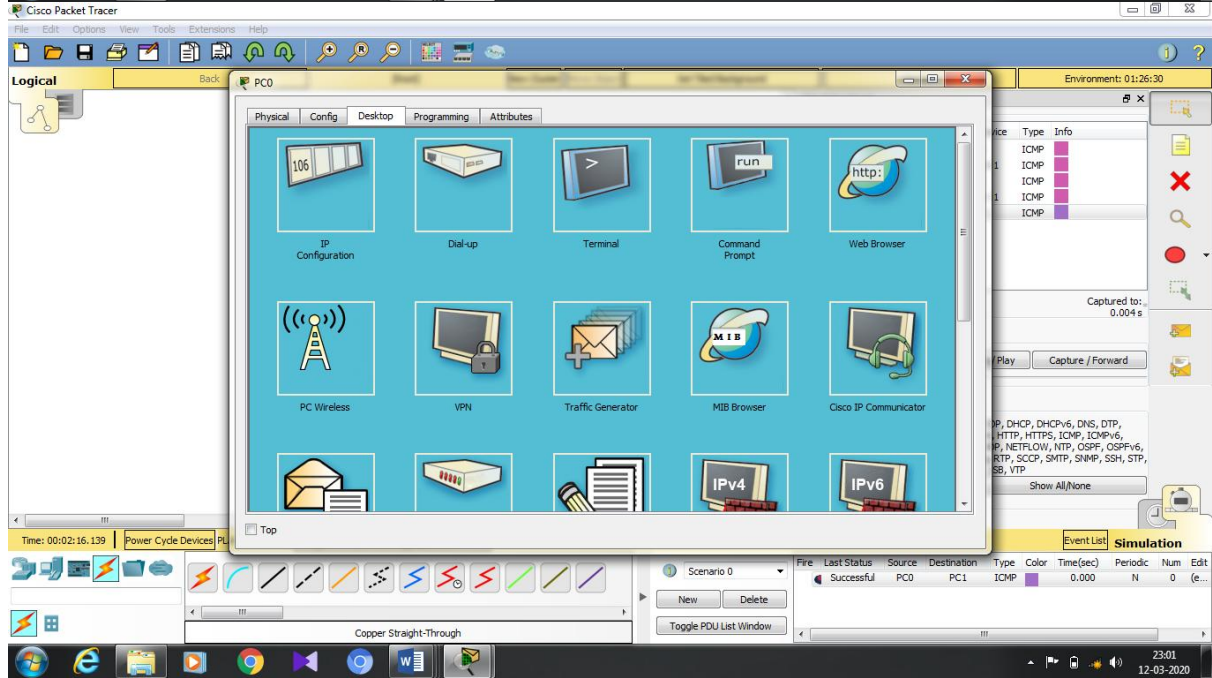
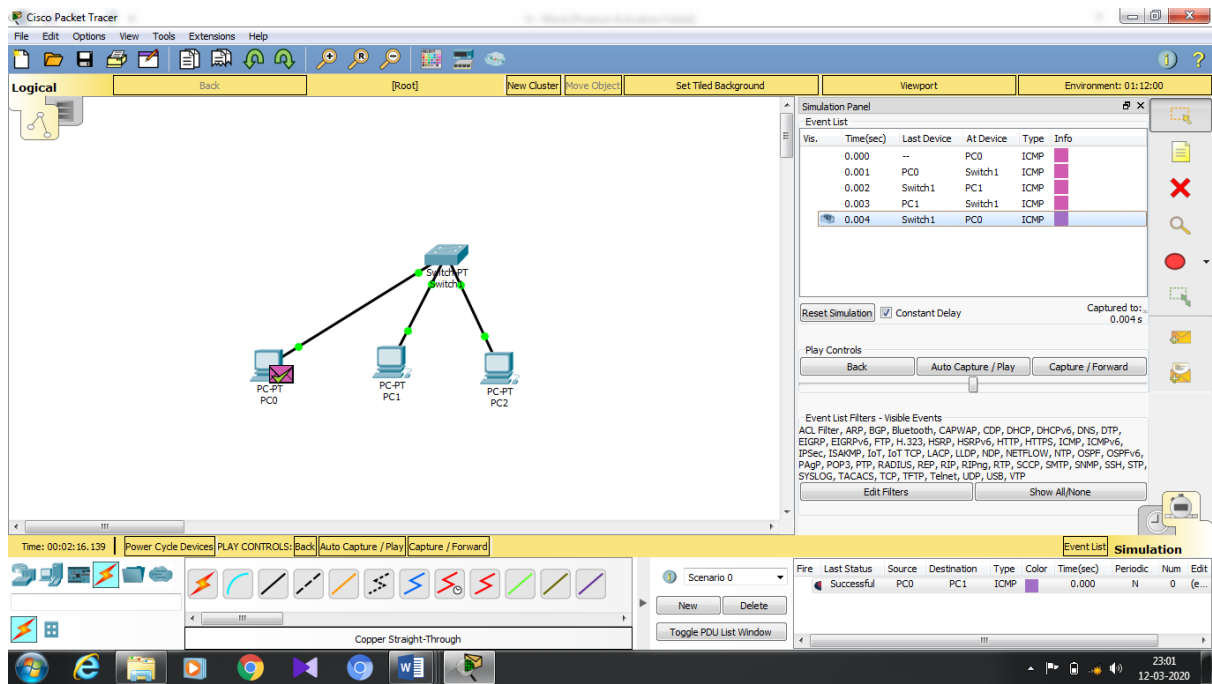
- Model the network topology specifying the nodes on the network and the links between those nodes
- Model the application flow (traffic) between the nodes

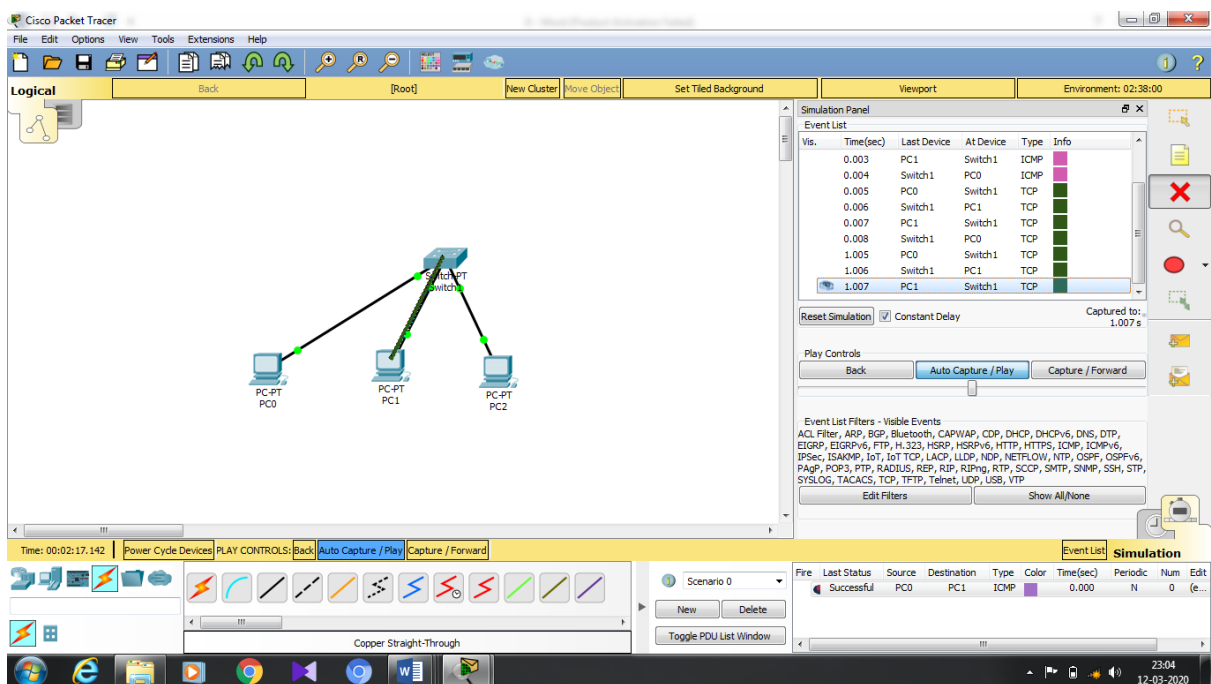
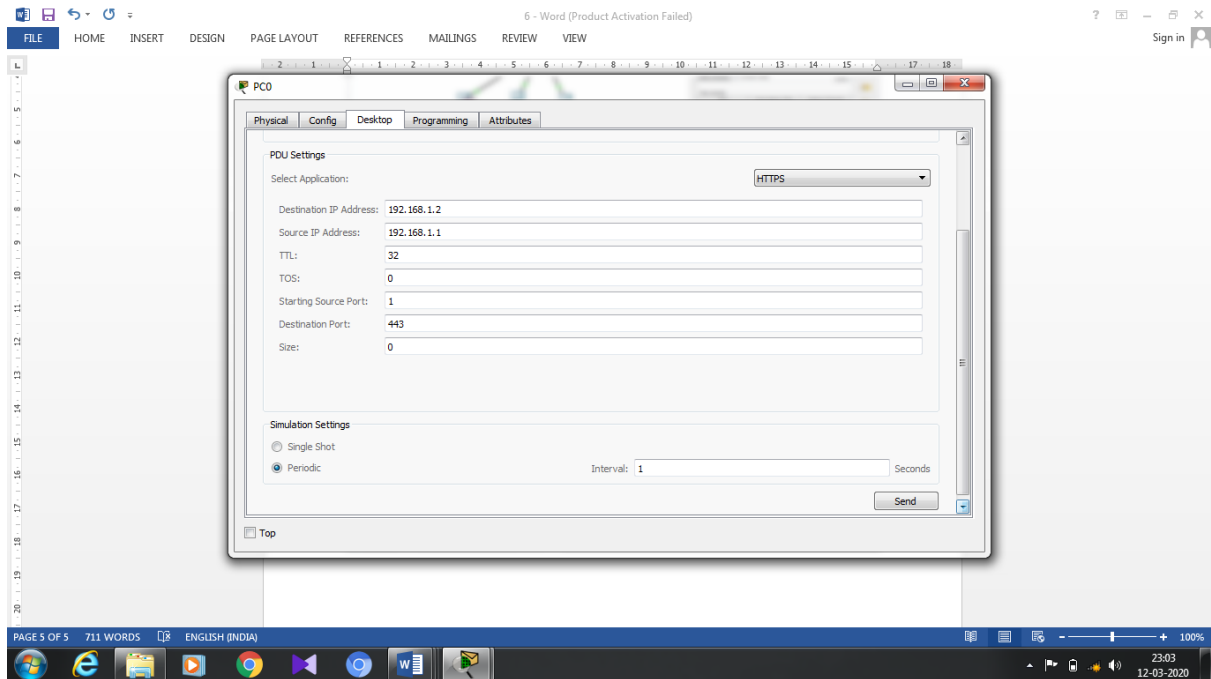
- Providing network performance metrics as output
- Visualization of the packet flow
- Technology / protocol evaluation and device designs
- Logging of packet/events for drill down analyses / debugging

Simulation of Congestion Control:









RESULT:

Thus the study of network simulator and congestion control packet tracer was executed successfully.

Ex.No:8 STUDY OF TCP/UDP PERFORMANCE USING SIMULATION TOOL

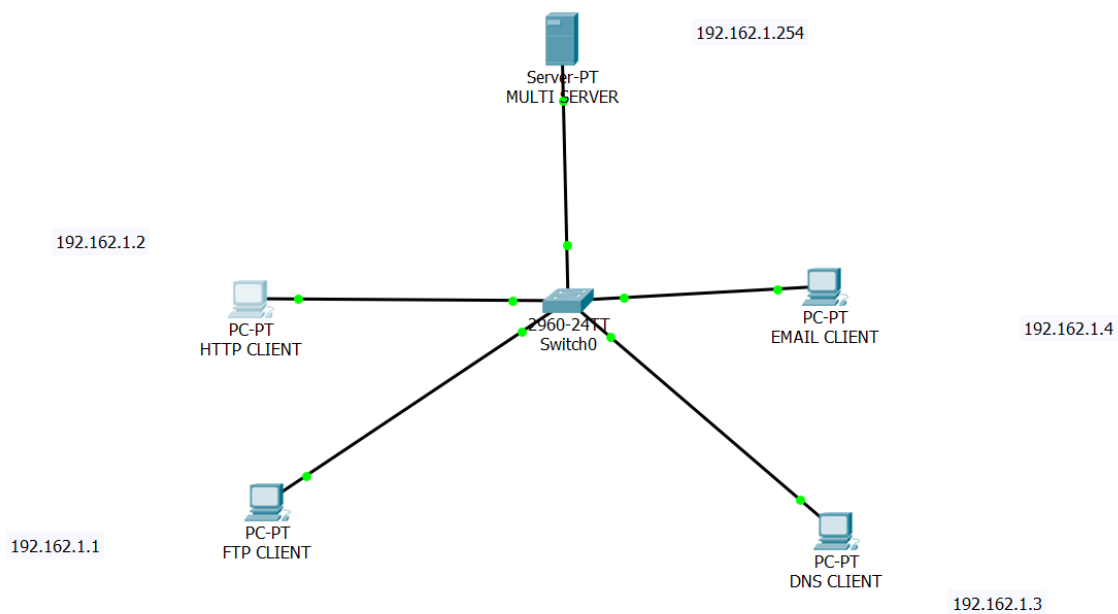
Aim:

To Study about TCP/UDP performance using Simulation tool.

Algorithm:

- i)Open Cisco packet tracer
- ii)Place Switch, laptop, and pc and assign IP address for laptop and PC
- iii)No default gateway is needed because there is no router
- iv)Click server ,Servicesand select services
- v)Enter thedomain name and Username,password
- vi)Click Email server enter name and address

Steps:



HTTP CLIENT

PhysicalConfigDesktopProgrammingAttributes

IP Configuration X

IP Configuration

☐ DHCP

☒ Static

IP Address

192.162.1.2

Subnet Mask

255.255.255.0

Default Gateway

0.0.0.0

DNS Server

192.162.1.254

IPv6 Configuration

☐ DHCP

☐ Auto Config

☒ Static

IPv6 Address

 /

Link Local Address

FE80::204:0AFF:FE1A:8541

FTP CLIENT

PhysicalConfigDesktopProgrammingAttributes

IP Configuration

IP Configuration

☐ DHCP

☒ Static

IP Address

192.162.1.1

Subnet Mask

255.255.255.0

Default Gateway

0.0.0.0

DNS Server

192.162.1.254

DNS CLIENT

PhysicalConfigDesktopProgrammingAttributes

IP Configuration X

IP Configuration

☐ DHCP

☒ Static

IP Address

192.162.1.3

Subnet Mask

255.255.255.0

Default Gateway

0.0.0.0

DNS Server

192.162.1.254

IPv6 Configuration

EMAIL CLIENT

Physical Config Desktop Programming Attributes

IP Configuration X

IP Configuration

☐ DHCP ☒ Static

IP Address 192.162.1.4

Subnet Mask 255.255.255.0

Default Gateway 0.0.0.0

DNS Server 192.162.1.254

CONFIGURATION IN MULTI SERVER

MULTI SERVER

Physical Config Services Desktop Programming Attributes

SERVICES

HTTP

DHCP

DHCPv6

TFTP

DNS

SYSLOG

AAA

NTP

EMAIL

FTP

IoT

VM Management

HTTP

HTTP ☒ On ☐ Off

HTTPS ☒ On ☐ Off

File Manager

	File Name	Edit	Delete
1	copyrights.html	(edit)	(delete)
2	cscoptlogo177x111.jpg		(delete)
3	helloworld.html	(edit)	(delete)
4	image.html	(edit)	(delete)
5	index.html	(edit)	(delete)

MULTI SERVER

PhysicalConfigServicesDesktopProgrammingAttributes

SERVICES

HTTP

DHCP

DHCPv6

TFTP

DNS

SYSLOG

AAA

NTP

EMAIL

FTP

IoT

VM Management

EMAIL

SMTP Service

☒ ON
☐ OFF

POP3 Service

☒ ON
☐ OFF

Domain Name:

multiserver.pt.ptu

Set

User Setup

User

kabil

gs

Password

+

-

Change

Password

MULTI SERVER

PhysicalConfigServicesDesktopProgrammingAttributes

SERVICES

HTTP

DHCP

DHCPv6

TFTP

DNS

SYSLOG

AAA

NTP

EMAIL

FTP

IoT

VM Management

DNS

DNS Service

☒ On
☐ Off

Resource Records

Name

Type

A Record

Address

Add

Save

Remove

No.	Name	Type	Detail
0	multiserver.pt.ptu	A Record	192.162.1.254

DNS SERVER CMD

```
C:\>nslookup multiserver.pt.ptu

Server: [192.162.1.254]
Address: 192.162.1.254

Non-authoritative answer:
Name:    multiserver.pt.ptu
Address: 192.162.1.254
```

EMAIL SERVER (CONFIGURATION) PW 123

The image shows two overlapping windows from the Cisco Packet Tracer application. The top window is the 'Email Server (CONFIGURATION)' interface, which has a light gray background and contains several input fields and buttons. The bottom window is the 'HTTP CLIENT' interface, which has a white background and displays a web browser view.

Email Server (CONFIGURATION) Fields:

- Your Name:
- Email Address:
- Server Information:
 - Incoming Mail Server:
 - Outgoing Mail Server:
- Logon Information:
 - User Name:
 - Password:
- Buttons: Save, Clear, Reset

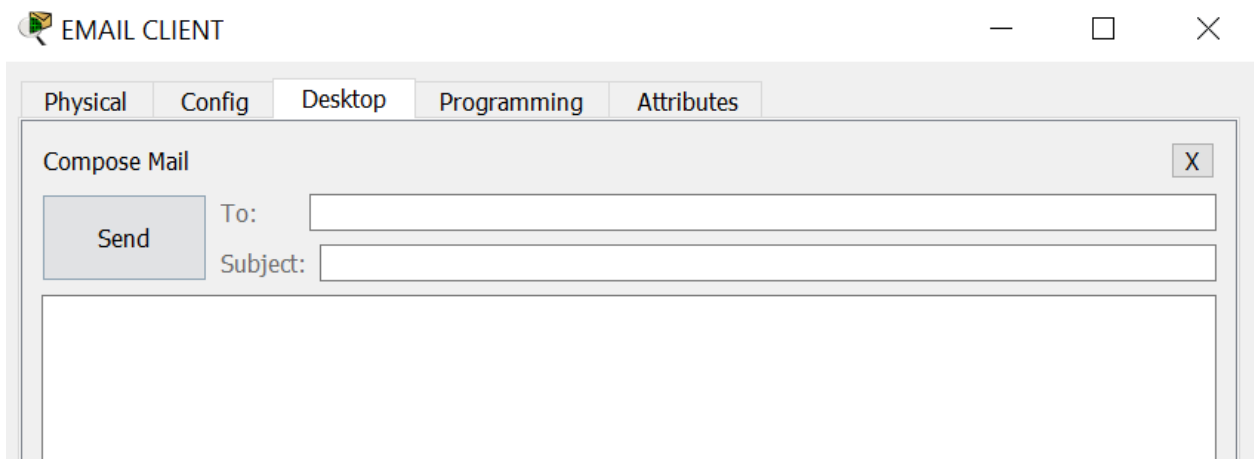
HTTP CLIENT Fields:

- Physical | Config | Desktop | Programming | Attributes
- Web Browser (Title Bar)
- Navigation: < > URL Go Stop
- Content: Cisco Packet Tracer
- Text: Welcome to Cisco Packet Tracer. Opening doors to new opportunities. Mind Wide Open.
- Quick Links:
 - [A small page](#)
 - [Copyrights](#)
 - [Image page](#)
 - [Image](#)

```
C:\>ftp 192.162.1.254
Trying to connect...192.162.1.254
Connected to 192.162.1.254
220- Welcome to PT Ftp server
Username:cisco
331- Username ok, need password
Password:
230- Logged in
(passive mode On)
ftp>quit
```

COMMANDS TO BE DONE WHILE TESTING

```
221- Service closing control connection.
C:\>ftp 192.162.1.254
Trying to connect...192.162.1.254
Connected to 192.162.1.254
220- Welcome to PT Ftp server
Username:cisco
331- Username ok, need password
Password:|
```



FOR TESTING (DO IT IN MULTISERVER) BROADCAST ADDRESS

```
C:\>ping 192.162.1.255

Pinging 192.162.1.255 with 32 bytes of data:

|
```

Result:

Thus the study of TCP/UDP perfrmane using simulation tool was executed successfully.

Ex.No.9a SIMULATION OF DISTANCE VECTOR ROUTING ALGORITHM

Aim:

To perform Simulation of Distance Vector Routing algorithm.

ALGORITHM:

1. Create a simulator object
2. Define different colors for different data flows
3. Open a nam trace file and define finish procedure then close the trace file, and execute nam on trace file.
4. Create n number of nodes using for loop
5. Create duplex links between the nodes
6. Setup UDP Connection between n(0) and n(5)
7. Setup another UDP connection between n(1) and n(5)
8. Apply CBR Traffic over both UDP connections
9. Choose distance vector routing protocol to transmit data from sender to receiver.
10. Schedule events and run the program.

Program (Distance Vector Protocol)

```
set ns [new Simulator]
set nr [open thro.tr w]
$ns trace-all $nr
set nf[open thro.nam w]
$ns namtrace-all $nf
proc finish { } {
    global ns nr nf
    $ns flush-trace
    close $nf
    close $nr
    exec nam thro.nam &
    exit 0
}
for { set i 0 } { $i < 12 } { incr i 1 } {
    set n($i) [$ns node]
    for {set i 0} { $i < 8 } {incr i} {
        $ns duplex-link $n($i) $n([expr $i+1]) 1Mb 10ms DropTail }
```

```
$ns duplex-link $n(0) $n(8) 1Mb 10ms DropTail
$ns duplex-link $n(1) $n(10) 1Mb 10ms DropTail
$ns duplex-link $n(0) $n(9) 1Mb 10ms DropTail
$ns duplex-link $n(9) $n(11) 1Mb 10ms DropTail
$ns duplex-link $n(10) $n(11) 1Mb 10ms DropTail
$ns duplex-link $n(11) $n(5) 1Mb 10ms DropTail
```

```
set udp0 [new Agent/UDP]
$ns attach-agent $n(0) $udp0
```

```
set cbr0 [new Application/Traffic/CBR]
$cbr0 set packetSize_ 500
$cbr0 set interval_ 0.005
$cbr0 attach-agent $udp0
```

```
set null0 [new Agent/Null]
$ns attach-agent $n(5) $null0
$ns connect $udp0 $null0
```

```
set udp1 [new Agent/UDP]
$ns attach-agent $n(1) $udp1
```

```
set cbr1 [new Application/Traffic/CBR]
$cbr1 set packetSize_ 500
$cbr1 set interval_ 0.005
$cbr1 attach-agent $udp1
```

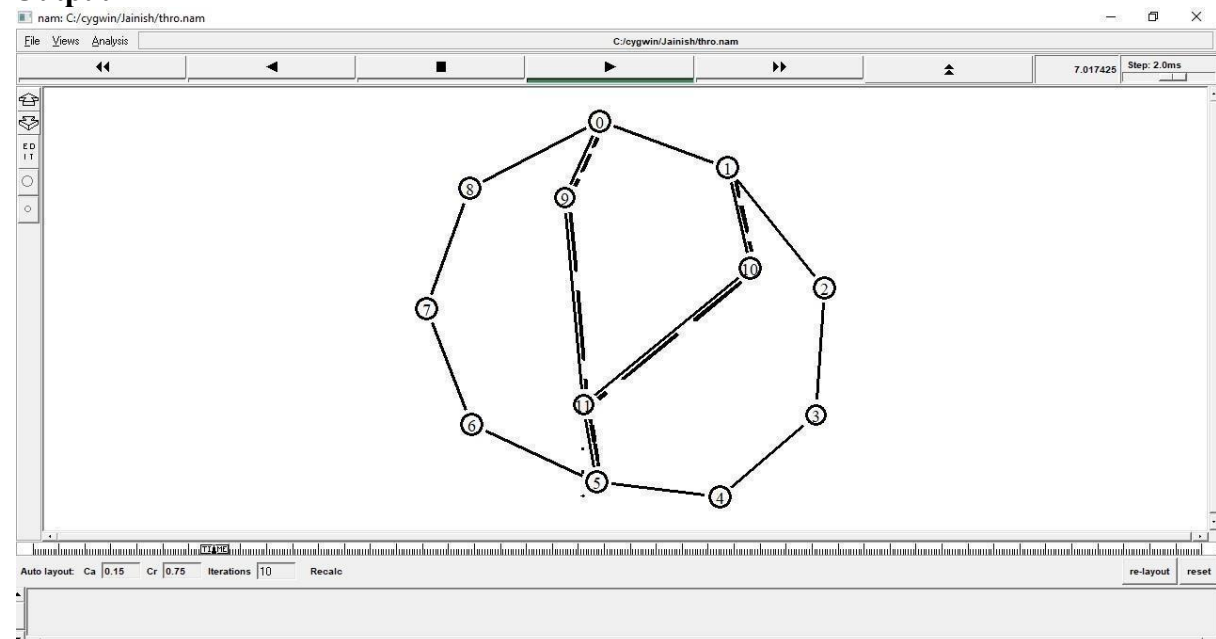
```
set null0 [new Agent/Null]
$ns attach-agent $n(5) $null0
$ns connect $udp1 $null0
```

```
$ns rtproto DV
```

\$ns rtmodel-at 20.0 up \$n(7) \$n(6)

\$ns color 2 Green

\$ns run



Thus performed Simulation of Distance Vector Routing algorithm.

Ex.No.9b**SIMULATION OF LINK STATE ROUTING ALGORITHM****Aim:**

To perform Simulation of Link State Routing algorithm.

Algorithm:

1. Create a simulator object
2. Define different colors for different data flows
3. Open a nam trace file and define finish procedure then close the trace file, and execute nam on trace file.
4. Create n number of nodes using for loop
5. Create duplex links between the nodes
6. Setup UDP Connection between n(0) and n(5)
7. Setup another UDP connection between n(1) and n(5)
8. Apply CBR Traffic over both UDP connections
9. Choose Link state routing protocol to transmit data from sender to receiver.
10. Schedule events and run the program.

Program (Link State Protocol)

```
set ns [new Simulator]
set nr [open link.tr w]
$ns trace-all $nr
set nf [open link.nam w]
$ns namtrace-all $nf
proc finish { } {
    global ns nr nf
    $ns flush-trace
    close $nf
    close $nr
    exec nam link.nam &
    exit 0
}
for { set i 0 } { $i < 12 } { incr i 1 } {
    set n($i) [$ns node]
    for {set i 0} { $i < 8 } {incr i} {
```

```
$ns duplex-link $n($i) $n([expr $i+1]) 1Mb 10ms DropTail }
```

```
$ns duplex-link $n(0) $n(8) 1Mb 10ms DropTail
```

```
$ns duplex-link $n(1) $n(10) 1Mb 10ms DropTail
```

```
$ns duplex-link $n(0) $n(9) 1Mb 10ms DropTail
```

```
$ns duplex-link $n(9) $n(11) 1Mb 10ms DropTail
```

```
$ns duplex-link $n(10) $n(11) 1Mb 10ms DropTail
```

```
$ns duplex-link $n(11) $n(5) 1Mb 10ms DropTail
```

```
set udp0 [new Agent/UDP]
```

```
$ns attach-agent $n(0) $udp0
```

```
set cbr0 [new Application/Traffic/CBR]
```

```
$cbr0 set packetSize_ 500
```

```
$cbr0 set interval_ 0.005
```

```
$cbr0 attach-agent $udp0
```

```
set null0 [new Agent/Null]
```

```
$ns attach-agent $n(5) $null0
```

```
$ns connect $udp0 $null0
```

```
set udp1 [new Agent/UDP]
```

```
$ns attach-agent $n(1) $udp1
```

```
set cbr1 [new Application/Traffic/CBR]
```

```
$cbr1 set packetSize_ 500
```

```
$cbr1 set interval_ 0.005
```

```
$cbr1 attach-agent $udp1
```

```
set null0 [new Agent/Null]
```

```
$ns attach-agent $n(5) $null0
```

```
$ns connect $udp1 $null0
```

```
$ns rtproto LS
```

\$ns rtmodel-at 10.0 down \$n(11) \$n(5)

\$ns rtmodel-at 15.0 down \$n(7) \$n(6)

\$ns rtmodel-at 30.0 up \$n(11) \$n(5)

\$ns rtmodel-at 20.0 up \$n(7) \$n(6)

\$udp0 set fid_1

\$udp1 set fid_2

\$ns color 1 Red

\$ns color 2 Green

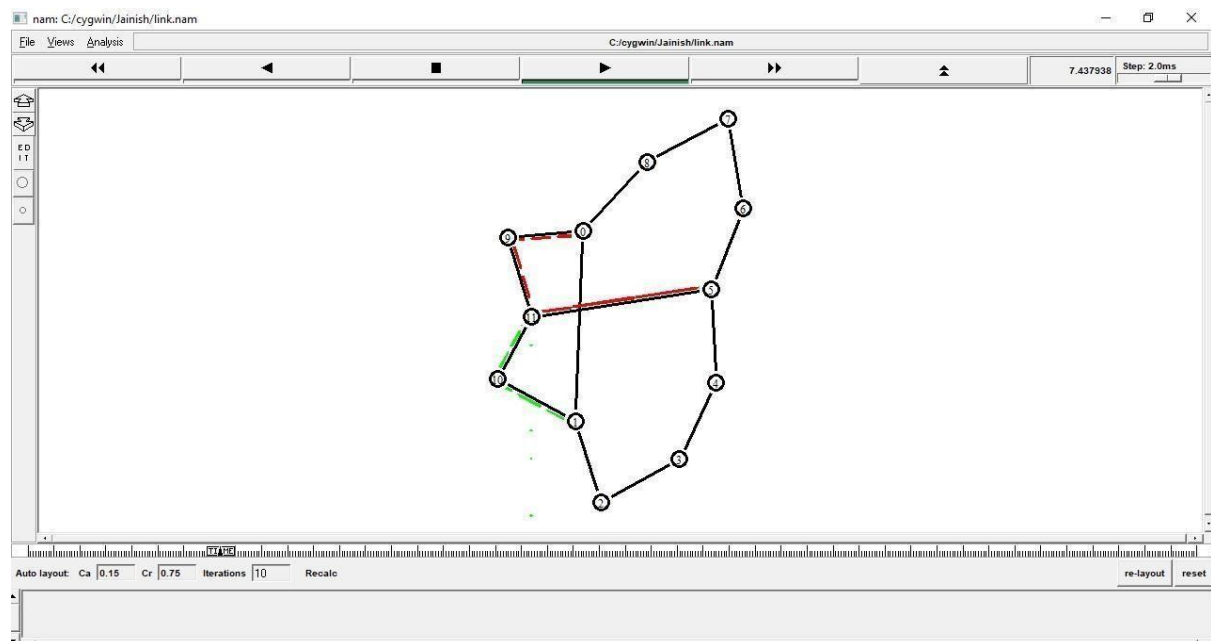
\$ns at 1.0 "\$cbr0 start"

\$ns at 2.0 "\$cbr1 start"

\$ns at 45 "finish"

\$ns run

Output:



Result:

Thus performed Simulation of Link State Routing algorithm.

Ex.No.10 SIMULATION OF ERROR CORRECTION CODE (LIKE CRC)

Aim :

To write a program in Java to implement the Simulation of Error Correction Code (CRC)

Algorithm :

At sender side

1. Start the program
2. Read the number of bits to be sent. Let n be the Number of bits in data to be sent from sender side.
3. Read the number of bits in the divisor. Let k be the Number of bits in the divisor (key obtained from generator polynomial).
4. The binary data is first increased by adding k-1 zeros in the end of the data
5. Use modulo-2 binary division to divide binary data by the divisor and store remainder of division.
6. Append the remainder at the end of the data to form the encoded data and send the same

At receiver side

1. Perform modulo-2 division again and if remainder is 0, then there are no errors.

Modulo 2 division

- In each step, a copy of the divisor (or data) is XORed with the k bits of the dividend.
- The result of the XOR operation (remainder) is (n-1) bits, which is used for the next step after 1 extra bit is pulled down to make it n bits long.
- When there are no bits left to pull down, we have a result. The (n-1)-bit remainder which is appended at the sender side.

Program

```
import java.io.*;
import java.util.*;
class crc
{
    public static void main(String args[])
    {
        Scanner input = new Scanner(System.in);
        int n;
```

```

// Read input
System.out.println("Enter the size of the data:");
n = input.nextInt();
int data[] = new int[n];
System.out.println("Enter the data, bit by bit:");
for(int i=0 ; i < n ; i++)
{
    System.out.println("Enter bit number " + (n-i) + ":");
    data[i] = input.nextInt();
}

//Read Divisor
System.out.println("Enter the size of the divisor:");
n = input.nextInt();
int divisor[] = new int[n];
System.out.println("Enter the divisor, bit by bit:");
for(int i=0 ; i < n ; i++)
{
    System.out.println("Enter bit number " + (n-i) + ":");
    divisor[i] = input.nextInt();
}

//Perform Division
int remainder[] = divide(data, divisor);
for(int i=0 ; i < remainder.length-1 ; i++)
{
    System.out.print(remainder[i]);
}
System.out.println("\nThe CRC code generated is:");

for(int i=0 ; i < data.length ; i++)
{
    System.out.print(data[i]);
}
for(int i=0 ; i < remainder.length-1 ; i++)

```

```

        {
            System.out.print(remainder[i]);
        }
        System.out.println();
        // Append the remainder
        int sent_data[] = new int[data.length + remainder.length - 1];
        System.out.println("Enter the data to be sent:");
        for(int i=0 ; i < sent_data.length ; i++)
        {
            System.out.println("Enter bit number " + (sent_data.length-i) + ":");
            sent_data[i] = input.nextInt();
        }
        receive(sent_data, divisor);
    }
    static int[] divide(int old_data[], int divisor[])
    {
        int remainder[] , i;
        int data[] = new int[old_data.length + divisor.length];
        System.arraycopy(old_data, 0, data, 0, old_data.length);

        // Remainder array stores the remainder
        remainder = new int[divisor.length];

        // Initially, remainder's bits will be set to the data bits
        System.arraycopy(data, 0, remainder, 0, divisor.length);
        for(i=0 ; i < old_data.length ; i++)
        {
            System.out.println((i+1) + ".) First data bit is : "
                                + remainder[0]);

            System.out.print("Remainder : ");

            // If first bit of remainder is 1 then exor the remainder bits with divisor bits

            if(remainder[0] == 1)
            {

```

```

        for(int j=1 ; j < divisor.length ; j++)
        {
            remainder[j-1] = exor(remainder[j], divisor[j]);
            System.out.print(remainder[j-1]);
        }
    }
    else
    {
        // If first bit of remainder is 0 then exor the remainder bits with 0
        for(int j=1 ; j < divisor.length ; j++)
        {
            remainder[j-1] = exor(remainder[j], 0);
            System.out.print(remainder[j-1]);
        }
    }
    remainder[divisor.length-1] = data[i+divisor.length];
    System.out.println(remainder[divisor.length-1]);
}
return remainder;
}
static int exor(int a, int b)
{
    if(a == b)
    {
        return 0;
    }
    return 1;
}
static void receive(int data[], int divisor[])
{
    int remainder[] = divide(data, divisor);
    for(int i=0 ; i < remainder.length ; i++)
    {

```

```

        if(remainder[i] != 0)
        {
            System.out.println("There is an error in received data...");
            return;
        }
    }
    System.out.println("Data was received without any error.");
}
}

```

Output

D:\Abarna>javac crc.java

D:\Abarna>java crc

Enter the size of the data: 7

Enter the data, bit by bit:

Enter bit number 7:

1

Enter bit number 6:

0

Enter bit number 5:

0

Enter bit number 4:

1

Enter bit number 3:

1

Enter bit number 2:

0

Enter bit number 1:

1

Enter the size of the divisor: 4

Enter the divisor, bit by bit:

Enter bit number 4:

1

Enter bit number 3:

0

Enter bit number 2:

1

Enter bit number 1:

1

1.) First data bit is : 1

Remainder : 0101

2.) First data bit is : 0

Remainder : 1010

3.) First data bit is : 1

Remainder : 0011

4.) First data bit is : 0

Remainder : 0110

5.) First data bit is : 0

Remainder : 1100

6.) First data bit is : 1

Remainder : 1110

7.) First data bit is : 1

Remainder : 1010

101

The CRC code generated is:

1001101101

Enter the data to be sent:

Enter bit number 10:

1

Enter bit number 9:

0

Enter bit number 8:

0

Enter bit number 7:

1

Enter bit number 6:

1

Enter bit number 5:

0

Enter bit number 4:

1

Enter bit number 3:

1

Enter bit number 2:

0

Enter bit number 1:

1

1.) First data bit is : 1

Remainder : 0101

2.) First data bit is : 0

Remainder : 1010

3.) First data bit is : 1

Remainder : 0011

4.) First data bit is : 0

Remainder : 0111

5.) First data bit is : 0

Remainder : 1110

6.) First data bit is : 1

Remainder : 1011

7.) First data bit is : 1

Remainder : 0000

8.) First data bit is : 0

Remainder : 0000

9.) First data bit is : 0

Remainder : 0000

10.) First data bit is : 0

Remainder : 0000

Data was received without any error.

Result:

Thus a program in Java implemented the Simulation of Error Correction Code (CRC)